```
function findClosestEntryInNodesList(nodes,
targetNode)                          #1
  closestEntry ←
null                                                    #2
  closestNode ← null
  for node in nodes
do                                                      #3
    closestEntryInNode ←
node.getClosestCentroidTo(targetNode)               #4
    if closerThan(closestEntryInNode, closestEntry,
targetNode) then #5
      closestEntry ←
closestEntryInNode                                      #6
      closestNode ←
node                                                    #7
  return (closestEntry,
closestNode)                                         #8
```

#1 Function `findClosestEntryInNodesList` takes a list of nodes and a target node, and returns the closest entry to the target and the node in the list that contains it. An entry here is, again, meant as either a point (if `nodes` are leaves) or a child node (if `nodes` contains internal nodes) The definition of “closest” is encapsulated in the two auxiliary methods called at lines #4 and #5.

#2 Initialize the results to `null`; it is assumed that at line #5 function `closerThan` will return the first argument, when `closestEntry` is `null`.

#3 Cycle through all the nodes in the input list.

#4 For each node, get its closest entry to `targetNode`. By default, closest can be meant as “with minimal Euclidean distance”.

#5 Compare the entry just computed to the best result found so far.